Code : LGLSI-3-26-94

Republic of Tunisia
Ministry of Higher Education and Scientific Research

University of Monastir
Faculty of Sciences of Monastir

End-of-studies Internship Report
for obtaining the

Bachelor's Degree in Software Engineering and Information Systems

Subject: Perf.Analysis counsulting

Developed by: HAMMED AMEN ALLAH

Defended on 13/06/2026 at 09:30 at the Amphi 3 room in front of the Jury:

President: Mohamed Neji Maatouk
Rapporteur: Salma Chaieb
University supervisor: Mohsen Maraoui
Professional Supervisor: Sghaier Oussama

Performed at: Perf-analysis counsulting

Academic Year 2025/2026

Acknowledgments

This work was completed at Perf-Analysis Consulting, and the author would like to express sincere gratitude to all contributors.

Special thanks are addressed to Mr. Oussama Sghaier, Company Supervisor, for his guidance, availability, and continuous technical support throughout the internship.

The author also expresses appreciation to Mr. Mohsen Maraoui, FSM Supervisor, for his constructive feedback, academic follow-up, and valuable recommendations during the preparation of this report.

Grateful acknowledgment is extended to the Faculty of Sciences of Monastir and to all team members who supported the successful completion of this project.

Abstract

Football clubs increasingly rely on digital tools to manage documents, events, staff coordination, player information, and operational communication. However, these activities are often distributed across independent tools, which creates fragmented data, limited traceability, and inconsistent access-control practices. This project proposes Football OS, a unified digital workspace designed to centralize key football club operations while preserving modularity and controlled access. The solution combines an Angular frontend with a Spring Boot backend structured around an API Gateway, a Governance Service, a Workspace Service, and a Shared SDK. The platform supports secured access, role-based authorization, document management, calendar and event workflows, player profile consultation, notifications, search, and online document viewing and editing through OnlyOffice. The result is a modular platform foundation that improves operational organization, supports maintainability, and provides a basis for future football-specific workflow extensions.

Keywords: Football OS; Microservices-Oriented Architecture; API Gateway; Role-Based Access Control; OIDC; JWKS; Document Management; Spring Boot.

Contents

List of Figures

List of Tables

List of Abbreviations

The following abbreviations are used throughout this report:

API Application Programming Interface

JWT JSON Web Token

OIDC OpenID Connect

JWKS JSON Web Key Set

RBAC Role-Based Access Control

SOA Service-Oriented Architecture (classic comparison model)

SDK Software Development Kit

OPA Open Policy Agent

SSO Single Sign-On

UML Unified Modeling Language

UI User Interface

UX User Experience

General Introduction

Context

Football clubs increasingly depend on digital systems to manage operational activities such as documents, events, staff coordination, player information, and internal communication. In practice, these activities are often distributed across independent tools with different data models and permission mechanisms. This situation reduces operational coherence and complicates controlled collaboration between administrative, technical, and medical stakeholders.

Problem Statement

The central problem addressed in this project is the following: how to design a unified and modular platform that centralizes football club operations while ensuring traceability, consistent access control, and maintainable service separation. Existing tool fragmentation generally leads to scattered data, limited audit visibility, and heterogeneous authorization practices that are not aligned with club roles and responsibilities.

Project Objectives

The project objective is to design and implement **Football OS** as a structured operational workspace based on a microservices-oriented architecture. The solution combines an Angular frontend with a Spring Boot backend organized around an API Gateway, a Governance Service, a Workspace Service, and a Shared SDK.

The SMART-oriented objectives adopted for this work are summarized below:

1. Centralize operational documents and events in a unified workspace model.
2. Secure access through authentication and role-based authorization rules.
3. Provide player profile consultation filtered by granted permissions.
4. Integrate online document viewing and editing workflows.
5. Structure backend responsibilities around Gateway, Governance, Workspace, and Shared SDK modules.

Adopted Methodology

The work followed an iterative incremental approach organized around successive technical layers: backend foundation, governance mechanisms, workspace feature implementation, frontend integration, and validation. Each iteration delivered an integrated subset of capabilities and then consolidated routing, authorization, data handling, and user workflows before extending the scope.

Report Organization

This report is organized into five main chapters. Chapter 1 presents the internship framework and the existing operational context. Chapter 2 details the requirement specification through actors, functional and non-functional requirements, and use cases. Chapter 3 presents the design choices, including architecture, modules, data design, and processing flows. Chapter 4 describes the implementation and realization of the solution, including interfaces and deployment configuration. Chapter 5 presents the testing and validation activities performed to assess the implemented platform. The report ends with a general conclusion.

Chapter 1: Internship Framework Presentation

This chapter defines the internship framework and the operational context of the project. It presents the host company, analyzes the existing environment, describes the adopted development methodology, and introduces the proposed solution.

1.1 Company Presentation

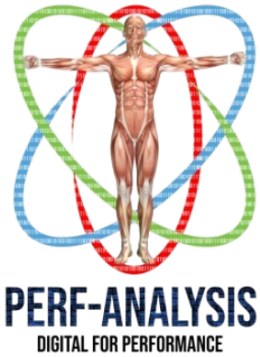

Figure 1.1: Perf Analysis Consulting logo

Perf Analysis Consulting develops digital tools for professional football organizations. The internship was conducted within the software engineering activity of the company and focused on **Football OS**, with emphasis on backend foundation services, governance mechanisms, and the Workspace module.

1.2 Existing Operational Environment Analysis

1.2.1 Current Environment

In the current operational environment, football club activities are generally supported by separate general-purpose tools. Communication, file sharing, calendar management, and player-related data handling are performed across different platforms that are not designed as a unified club workspace.

The company also had a previous internal solution centered mainly on file storage. Although useful for basic document handling, it did not provide a complete operational model including event workflows, structured notifications, governance-driven permissions, and unified auditability.

1.2.2 Comparative View of Existing Solutions

Table 1.1 summarizes the principal tools observed in practice, their contributions, and their limits with respect to a unified operational platform.

Table 1.1: Comparison of existing solutions and identified gaps

Existing solution	**Main limitation**	**Football OS response**
WhatsApp (team messaging)	Communication is fast but weakly structured and hard to audit.	Connects communication with events, documents, and traceable actions.
Cloud drives (Dropbox/Drive)	File sharing is simple but weakly linked to governance and event workflows.	Adds role-based permissions and governed document lifecycle.
Google Calendar	Planning is convenient but not integrated with tasks, policy checks, and required documents.	Provides unified event workflow with attendees, tasks, documents, and notifications.
Excel sheets	Data entry is flexible but consolidation and access control are inconsistent.	Enables controlled player-data consultation with permission-filtered visibility.
Previous internal file-storage solution	Scope was mostly limited to storage, without unified governance, notifications, workflow orchestration, and auditability.	Delivers an end-to-end platform combining governance and workspace services.

Table 1.1 shows that the main issue is not the absence of tools, but the absence of a coherent operational architecture that unifies identity, authorization, canonical data, and workflow traceability.

1.3 Development Methodology

The project followed an iterative incremental approach. Each iteration produced an integrated deliverable, then refined technical consistency before extending scope. This cadence made it possible to build foundational services first and add operational features progressively. The Shared SDK was developed as a dedicated standalone foundation iteration before service feature iterations.

Table 1.2: Iterative delivery plan used during development

Iteration	Objective	Main Deliverables
Iteration 0	Shared SDK foundation	Standalone SDK module, shared contracts, security/context helpers, and reusable integration clients consumed by backend services.
Iteration 1	Platform foundation	API Gateway routing baseline, service structure, and Docker-based environment.
Iteration 2	Governance	Actors, roles, players, policy integration, and audit mechanisms.
Iteration 3	Workspace documents	Document upload, metadata handling, storage integration, and OnlyOffice connection.
Iteration 4	Calendar and events	Event management, attendees, tasks, and notification workflows.
Iteration 5	Integration and validation	Frontend integration, functional validation, and corrective refinements.

As shown in Table 1.2, the sequence progressed from SDK-first foundation and architectural baseline to governance controls, then to business workflows and final integration.

1.4 Proposed Solution Positioning

To address the identified gaps, the project proposes **Football OS** as a unified workspace for football club operations.

The platform is structured around two complementary layers:

- **Governance layer:** identity context, roles, permissions, canonical players and teams, and audit traceability.
- **Workspace layer:** documents, events, notifications, search, and player profile consultation.

This structure is supported by a modular architecture that separates concerns between Gateway, Governance, Workspace, and Shared SDK modules. The separation improves maintainability and enables controlled future extension without restructuring the entire platform.

Conclusion

This chapter established the internship context, analyzed the existing operational environment, and positioned Football OS as a structured response to identified coordination and governance gaps. The next chapter translates this context into formal functional and non-functional requirements.

Chapter 2: Specification of Needs

Introduction

This chapter formalizes the needs of Football OS through measurable requirements, actor definitions, and use-case models. The focus is on expected system behavior and access rules, independently from implementation details.

2.1 Functional Requirements

Functional requirements are summarized in Table 2.1, with identifiers, actors, priorities, and delivery status in the current project scope.

Table 2.1: Functional requirements of Football OS

ID	Requirement	Main Actor	Priority	Status
FR-01	Authenticate through an external identity provider and enforce protected access.	All platform users	High	Implemented
FR-02	Manage users and roles for administrative governance.	Club Administrator	High	Partial
FR-03	Manage canonical club data (players and teams) with controlled updates.	Club Administrator	High	Partial
FR-04	Upload, consult, and archive documents under authorization rules.	Authorized staff	High	Implemented
FR-05	Support document versioning and online viewing or editing.	Authorized staff	High	Implemented
FR-06	Manage calendar events, attendees, and task coordination.	Head Coach / Club Administrator	High	Implemented
FR-07	Consult player profiles with role-based filtering.	Authorized staff	High	Partial
FR-08	Generate and consult notifications for operational updates.	Authorized staff	Medium	Implemented
FR-09	Search workspace resources with permission filtering.	Authorized staff	Medium	Implemented
FR-10	Enforce access control and activity tracking for accountability.	Backend services	High	Implemented

Table 2.1 reflects the current project baseline without overclaiming full maturity of every

feature.

2.2 Non-Functional Requirements

Non-functional requirements are formalized in Table 2.2 to clarify quality expectations and their validation approach.

Table 2.2: Non-functional requirements of Football OS

ID	Quality Attribute	Requirement	Validation Method
NFR-01	Security	Enforce authenticated access, role-based authorization, and protected sensitive exchanges.	Security tests, policy checks, and access-control cases.
NFR-02	Usability	Provide clear daily workflows with understandable user feedback.	UI scenario reviews and user acceptance checks.
NFR-03	Performance	Keep response times acceptable for common operations such as documents, events, and search.	Workflow-oriented performance checks.
NFR-04	Maintainability	Preserve modular separation of concerns and reusable cross-service contracts.	Architecture review and module inspections.
NFR-05	Reliability	Maintain consistent behavior and controlled failure handling in normal operations.	Integration and error-handling validation.
NFR-06	Traceability	Record significant operations for audit and accountability.	Audit-log verification and end-to-end trace checks.

2.3 Use Case Modeling

The use-case and actor models in this chapter follow UML notation conventions for behavioral modeling and diagram semantics [16].

2.3.1 Actor Model

Actors are consolidated in Table 2.3.

Table 2.3: Actor model of Football OS

Actor	Type	Role in the system
Club Administrator	Human	Manages users, roles, governance data, and high-level operational permissions.
Head Coach	Human	Manages events and operational coordination for football activities.
Coaching Staff	Human	Consults events, documents, and player-related operational information.
Medical Staff	Human	Accesses authorized medical and player-related information.
Identity Provider	Technical	Authenticates users and provides identity/session context.
Document Editor	Technical	Supports online document viewing and editing workflows.
Object Storage Provider	Technical	Stores document binary content and file versions.
Governance Policy Service	Technical	Evaluates authorization decisions for protected actions.
Canonical Player Data Provider	Technical	Provides canonical player identity references used by workspace features.

2.3.2 Global Platform Use Case Diagram

The global platform use-case diagram provides a high-level view of the interactions between Football OS and its actors. Figure 2.1 presents the global use-case view of Football OS. Human actors are represented as primary users of the platform, while technical actors represent external systems involved in authentication, policy evaluation, document editing, object storage, and canonical player data retrieval. The diagram summarizes the main services offered by the platform without exposing internal implementation details, since detailed processing flows are presented later in the design chapter.

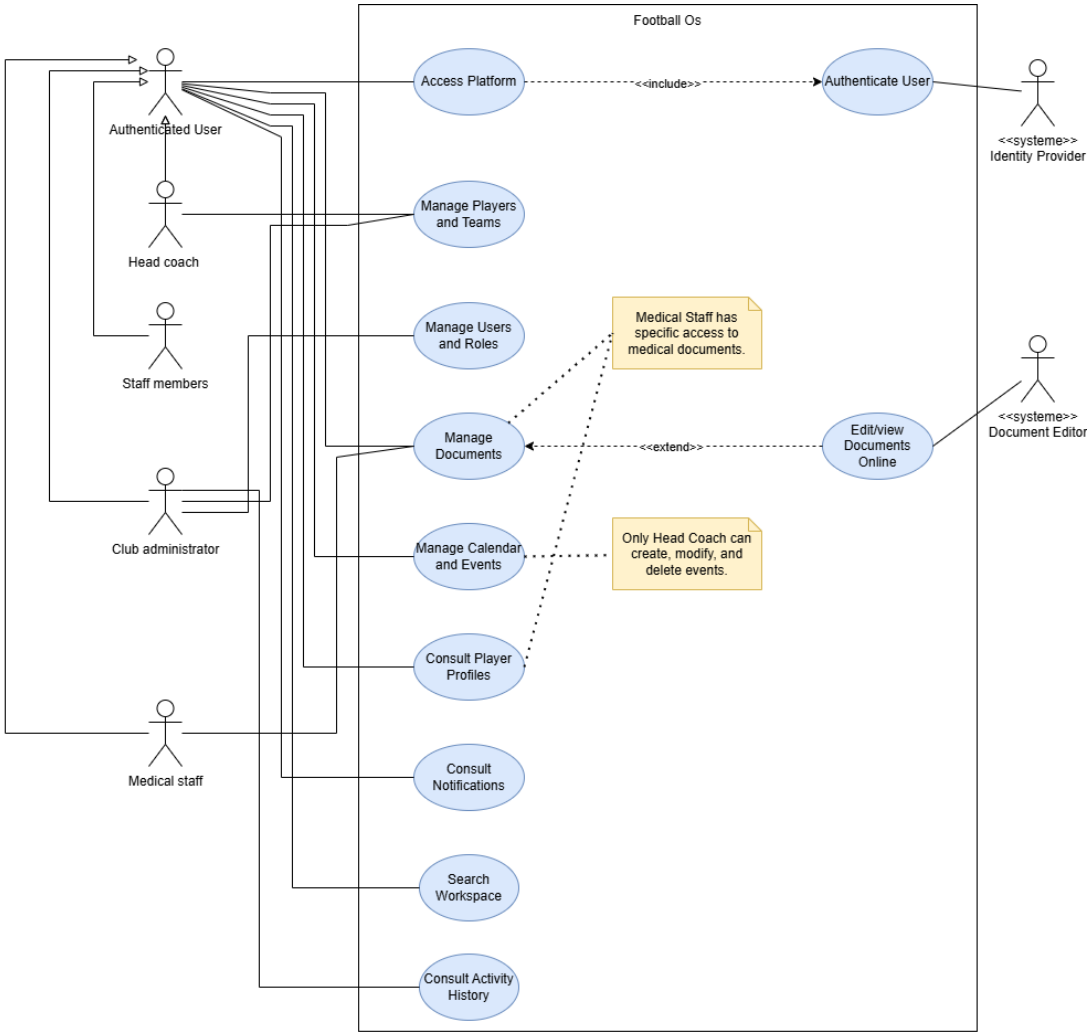

Figure 2.1: General use case diagram of the Football OS platform

The following use-case descriptions summarize the principal operational flows used in the current specification scope.

Table 2.4: Use-case description: Access Football OS Platform

Use Case	Access Football OS Platform
Primary Actor	Club Administrator / Head Coach / Coaching Staff / Medical Staff
Secondary Actor	Identity Provider
Objective	Authenticate the user and grant access to authorized platform features.
Preconditions	User account exists and identity provider is reachable.
Main Scenario	1. The user opens the platform. 2. The system redirects authentication to the identity provider. 3. The user authenticates successfully. 4. The system resolves role context and grants authorized access.
Alternative / Exception Scenario	Authentication failure or invalid token; access is denied and the user remains outside protected features.
Business Rules / Access Rules	Protected features require authenticated identity. Visible modules depend on role permissions.
Postconditions	Authenticated session is established with role-scoped access rights.

Table 2.4 specifies the authentication-oriented platform-access use case.

2.3.3 Calendar and Event Management

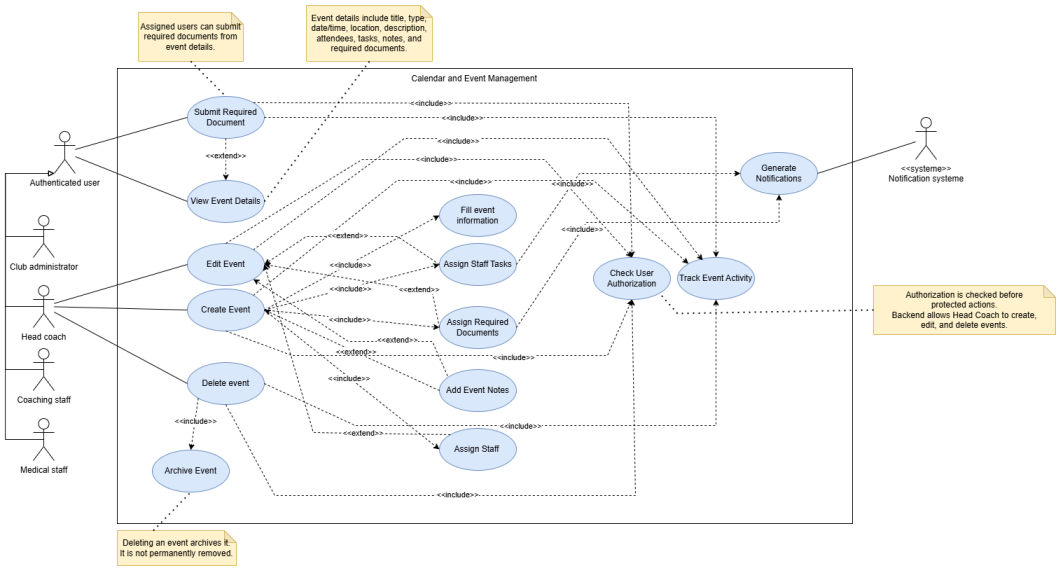

Figure 2.2: Calendar and event management use case diagram

Figure 2.2 illustrates the actor interactions for event planning, consultation, and archive operations.

Table 2.5: Use-case description: Calendar and Event Management

Use Case	Calendar and Event Management
Primary Actor	Head Coach / Club Administrator (management), authorized staff (consultation)
Objective	Plan, update, consult, and archive operational events and related assignments.
Preconditions	User is authenticated and has calendar access rights.
Main Scenario	1. The user opens the calendar module. 2. The user creates, updates, consults, or archives an event. 3. The system validates permissions and persists the operation. 4. The system updates related attendees, tasks, and notifications when applicable.
Alternative / Exception Scenario	Unauthorized update request, invalid event data, or unavailable service causes rejection with error feedback.
Business Rules / Access Rules	Event management actions are restricted to authorized roles. Delete behavior is treated as archive action, not hard deletion.
Postconditions	Event state is updated according to the accepted operation and authorized users are notified when required.

Table 2.5 formalizes the calendar and event use-case contract.

2.3.4 Document Management

This subsection defines document-management behavior and integration points through its use-case contract.

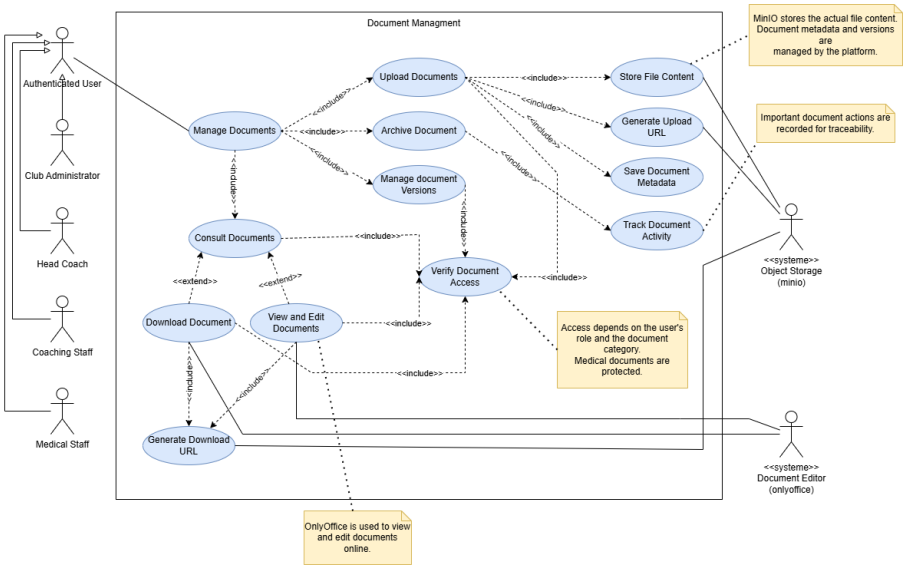

Figure 2.3: Document management use case diagram

Figure 2.3 illustrates the main actor interactions for document upload, consultation, editing, versioning, and archiving operations.

Table 2.6: Use-case description: Document Management

Use Case	Document Management
Primary Actor	Club Administrator / Head Coach / Coaching Staff / Medical Staff
Secondary Actors	Object Storage Provider / Document Editor
Objective	Manage document upload, consultation, versioning, editing, and archiving under controlled access.
Preconditions	User is authenticated and authorized for the requested document action.
Main Scenario	1. The user opens the document module and selects an action. 2. The system validates permissions and applies business rules. 3. The system stores or retrieves document metadata and versions. 4. The system invokes online editor integration when editing is requested.
Alternative / Exception Scenario	Upload failure, editor unavailability, or unauthorized access attempt triggers a controlled error response.
Business Rules / Access Rules	Document visibility depends on role and category. Medical documents are restricted to medical staff and explicitly authorized roles.
Postconditions	Document state and version history are updated according to authorized operations.

Table 2.6 defines the detailed scenarios and access constraints of document management.

2.3.5 Player Profile Consultation

Player profile consultation requires policy-driven visibility because sensitive medical and administrative information must remain restricted by role.

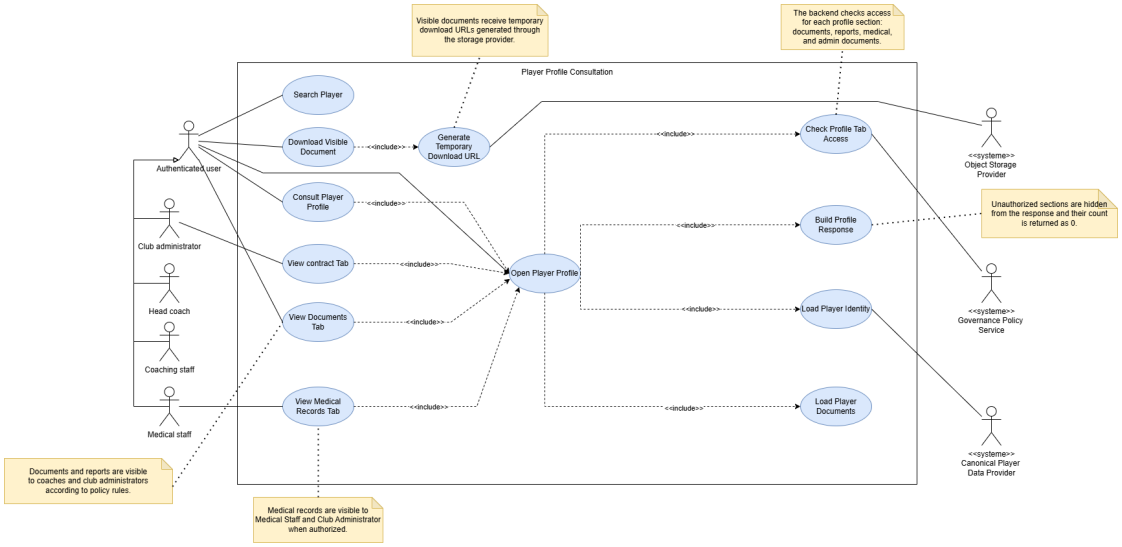

Figure 2.4: Player profile consultation use case diagram

Table 2.7: Use-case description: Consult Player Profile

Use Case	Consult Player Profile
Primary Actor	Club Administrator / Head Coach / Coaching Staff / Medical Staff
Secondary Actors	Canonical Player Data Provider / Governance Policy Service
Objective	Display player profile sections according to user permissions and available data.
Preconditions	User is authenticated and the selected player exists in reference data.
Main Scenario	1. The user selects a player profile. 2. The system retrieves canonical and workspace-linked information. 3. The system evaluates section-level permissions. 4. The system displays only authorized profile sections.
Alternative / Exception Scenario	Unknown player, unavailable linked data, or insufficient permission leads to partial display or access rejection.
Business Rules / Access Rules	Sensitive profile content, especially medical information, is visible only to authorized roles.
Postconditions	User receives a role-filtered profile view consistent with policy decisions.

Table 2.7 specifies the profile-consultation behavior under role-based restrictions.

Conclusion

This chapter defined functional and non-functional requirements, actor responsibilities, and principal use-case behaviors of Football OS. These specifications form the baseline for the architectural and design decisions in the next chapter.

Chapter 3: Design

Introduction

This chapter presents the design of Football OS and answers how the platform is organized and how the main processes are implemented. It focuses on the global architecture, backend module design, data design, processing design, and security and authorization design.

3.1 Global Architecture Design

3.1.1 Architectural Style and Decision Analysis

Football OS adopts a microservices-oriented architecture organized inside a monorepo. The system is not implemented as a single monolithic backend: the API Gateway, Governance Service, and Workspace Service are separated as distinct backend applications with their own responsibilities, runtime configuration, and persistence concerns. The monorepo structure was kept to simplify development, dependency management, and Shared SDK evolution during the internship.

This choice provides a compromise between a simple monolith and a fully distributed multi-repository microservices ecosystem. A monolith would have reduced deployment complexity but would have mixed governance, workspace operations, authorization, and document workflows inside one application. A fully independent multi-repository microservices organization would have increased operational overhead, CI/CD complexity, and version-management cost. The selected architecture keeps clear service boundaries while remaining manageable for a small-team academic project.

Classic SOA is retained in this report only as a comparison baseline, while the implemented backend aligns with a microservices-oriented monorepo model [17, 18]. Backend service internals still follow Onion/Clean Architecture layering to keep domain logic isolated from technical adapters [19].

The selected option is therefore not a classic centralized SOA model, but a microservices-oriented monorepo. Gateway owns edge enforcement, Governance owns identity/canonical/policy/audit concerns, Workspace owns operational document, event, profile, and notification workflows, and the Shared SDK reduces duplicated integration code while preserving service-level separation.

Table 3.1: Architecture decision matrix

Criterion	Monolith	Classic SOA	Multi-repository Microservices	Selected Microservices Monorepo
Maintainability	Medium	Medium	High	High
Deployment complexity	Low	Medium	High	Medium
Domain ownership	Low	Medium	High	High
Security isolation	Medium	Medium	High	High
Team-size suitability	High (small team)	Medium	Low (small team)	High (small team)
Latency overhead	Low	Medium	High	Medium
Shared code management	Simple but coupled	Moderate	Difficult across repos	Centralized and controlled

3.1.2 General Architecture

The following diagram shows how the frontend communicates with the backend through the Gateway, and how backend services rely on governance, workspace, SDK, and supporting infrastructure.

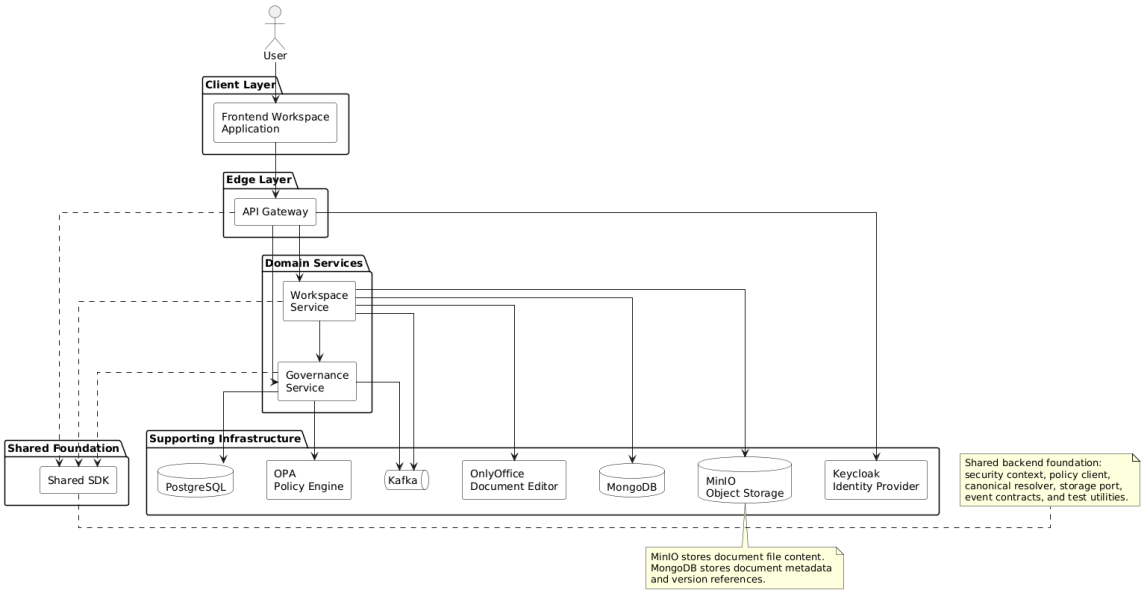

Figure 3.1: Global architecture of the Football OS platform

As shown in Figure 3.1, the frontend sends requests to the API Gateway, which routes calls to Governance or Workspace. The SDK provides shared abstractions, while infrastructure supports persistence, messaging, policy evaluation, storage, and document editing. The asynchronous communication design follows event-driven integration principles [23].

Table 3.2 clarifies that frontend API traffic goes through the Gateway rather than directly to

Table 3.2: Default local entry points used during development

Component	Port	Access role
Workspace Frontend	4200	User web interface.
API Gateway	8080	Public backend entry point for frontend API calls.
Governance Service	8081	Internal/backend endpoint behind Gateway routing.
Workspace Service	8082	Internal/backend endpoint behind Gateway routing.

internal services.

3.1.3 Main Components

Table 3.3 summarizes the main technical responsibilities and technologies used by each architecture component.

3.2 Backend Module Design

3.2.1 API Gateway

The API Gateway is the backend entry point. It validates token context, applies edge controls, and routes requests to Governance and Workspace without exposing internal service addresses.

3.2.2 Governance Service

The Governance Service owns actor/role management, canonical players/teams, policy decision support through OPA, and audit traceability data.

3.2.3 Workspace Service

The Workspace Service owns operational modules (documents, events, notifications, search, and profile aggregation), persists workspace aggregates in MongoDB, and integrates with Governance, MinIO, Kafka, and OnlyOffice.

3.2.4 Shared SDK

The Shared SDK is not a user-facing module. It is reused by backend services to avoid duplicated technical plumbing. It provides shared security context, policy client, canonical resolver, storage port abstraction, event contracts, audit hooks, and test utilities.

Table 3.3: Main components of the Football OS architecture

Component	Responsibility	Main Technology
Frontend Workspace Application	User interface for workspace operations and platform access.	Angular + Tailwind CSS
API Gateway	Entry point of backend APIs; routes requests and enforces secure access flow.	Spring Boot (Gateway layer)
Governance Service	Manages users, roles, permissions, canonical players/teams, policy decisions, and audit tracking.	Spring Boot + PostgreSQL + OPA
Workspace Service	Manages documents, calendar/events, notifications, search, and player profile aggregation.	Spring Boot + MongoDB + MinIO
Shared SDK	Shared foundation/library used by backend services; provides reusable abstractions and common integration mechanisms.	Java multi-module SDK
Keycloak / Identity Provider	Authenticates users and provides identity context.	Keycloak (OIDC/JWKS)
OPA Policy Engine	Evaluates authorization rules for protected actions and resources.	Open Policy Agent
Kafka	Supports asynchronous event messaging and notification propagation.	Apache Kafka
PostgreSQL	Stores governance structured data such as actors, roles, players, teams, and audit logs.	PostgreSQL
MongoDB	Stores workspace aggregates and document metadata/version references, including documents, events, and notifications.	MongoDB
MinIO	Stores the actual document file content and file versions.	MinIO Object Storage
Redis	Supports gateway-level rate limiting and request throttling.	Redis
OnlyOffice	Enables online document viewing and editing workflows.	OnlyOffice Docs

3.2.5 Service Boundary Summary

Table 3.4 consolidates the separation of responsibilities across backend modules.

3.2.6 API Summary

Table 3.5 provides a compact overview of representative endpoint families used by the platform.

3.3 Data Design

Table 3.4: Service boundary summary

Service / Module	Ownership	Persistence	Main integrations
API Gateway	Edge security, request enrichment, rate limiting, and routing to backend services.	No business DB	Keycloak JWKS, Redis, Governance Service, Workspace Service
Governance Service	Actors, roles, canonical players/teams, policy evaluation flow, and audit ownership.	PostgreSQL	OPA, Kafka, Keycloak, Shared SDK
Workspace Service	Documents, events, profiles, notifications, search, and OnlyOffice-related workflows.	MongoDB + MinIO	Governance policy/canonical APIs, Kafka, OnlyOffice
Shared SDK	Shared contracts and reusable adapters for security, policy, canonical, storage, events, and tests.	No business DB	Security, policy, canonical, storage, events, tests

3.3.1 Main Data Models

Football OS uses several services and storage technologies. For this reason, the data model is presented by domain instead of using one large global class diagram. Governance entities are persisted in PostgreSQL, workspace aggregates in MongoDB, and binary document content in MinIO.

3.3.2 Governance Relational Entities (PostgreSQL)

Table 3.6: Governance entities stored in PostgreSQL

Entity	Storage	Main fields	Purpose
Actor	PostgreSQL	id, identityRef, roleRefs, status	Represents platform actors and identity linkage.
Role	PostgreSQL	id, roleCode, permissions	Defines permission sets for authorization decisions.
Player	PostgreSQL	id, canonicalRef, names, teamRef	Maintains canonical player references used across modules.
Team	PostgreSQL	id, canonicalRef, name, metadata	Maintains canonical team references.
AuditLogEntry	PostgreSQL	id, actorRef, action, target, timestamp	Records traceable governance and access-related operations.

Table 3.6 summarizes the governance entities and their persistence responsibilities.

Table 3.5: API endpoint family summary

Module	Endpoint family	Purpose
Workspace	`/api/v1/documents`	Document upload, consultation, categorization, and lifecycle operations.
Workspace	`/api/v1/events`	Event creation, consultation, update, and archive workflows.
Workspace	`/api/v1/profiles`	Role-filtered player profile consultation and aggregation.
Workspace	`/api/v1/notifications`	Notification generation, retrieval, and state updates.
Workspace	`/api/v1/search`	Search across workspace resources with access filtering.
Workspace	`/api/v1/onlyoffice`	Online document editor integration and callback-related actions.
Governance	`/api/v1/actors`	Actor management and actor-role relationship management.
Governance	`/api/v1/players`	Canonical player data operations and references.
Governance	`/api/v1/teams`	Canonical team data operations and references.
Governance	`/api/v1/policy`	Authorization decision endpoints for protected actions.

3.3.3 Workspace Collections (MongoDB)

Table 3.7: Workspace collections stored in MongoDB

Collection	Stores	Main fields	Purpose
`workspace_documents`	Document metadata and versions	id, category, state, versionRefs, ownerRef	Supports document lifecycle and lookup.
`workspace_events`	Events and linked operational items	id, type, schedule, attendees, tasks, requiredDocuments	Supports event planning and tracking flows.
`workspace_notifications`	User notification entries	id, userRef, message, state, createdAt	Supports notification delivery and read-state tracking.

Table 3.7 summarizes the workspace collections used by operational modules.

3.3.4 Indexing and Consistency Strategy

Since Football OS separates governance data, workspace aggregates, and binary files across different storage systems, data access must rely on clear references and targeted indexes rather than cross-database foreign keys.

Table 3.8: Indexing and reference strategy

Storage	Index / Reference	Purpose
PostgreSQL Actor	identityRef, roleRefs	Fast actor lookup and role resolution.
PostgreSQL Player	canonicalRef, teamRef	Player and team reference lookup.
MongoDB `workspace_documents`	category, state, ownerRef	Document filtering by visibility, ownership, and lifecycle state.
MongoDB `workspace_events`	`schedule.startDate`, `attendees.actorRef`	Calendar listing and attendee-based event lookup.
MongoDB `workspace_notifications`	userRef, state, createdAt	Inbox retrieval and unread notification queries.
MinIO object keys	bucket, objectKey, version reference	Binary file retrieval without storing large files directly in MongoDB or PostgreSQL.

Cross-service references are stored as identifiers rather than relational foreign keys across databases. Governance remains the source of truth for actors, roles, players, and teams, while Workspace stores references to these canonical entities. When canonical data is required, Workspace resolves it through Governance and SDK integration instead of duplicating governance state. This approach reduces coupling between databases, but it also requires careful handling of unavailable references and eventual consistency between services.

3.3.5 Governance Service Design

The Governance Service acts as the control plane of Football OS. It centralizes identity management, canonical football entities, authorization decisions, signal processing, and audit traceability. The first governance diagram in this section is intentionally a package-level dependency view, not a class-level model. It provides a global structural perspective before zooming into selected internal packages.

Governance Service Package Dependency Diagram

The governance codebase is organized into feature packages: `canonical`, `identity`, `policy`, `signal`, and `config`. Inside `canonical`, `identity`, `policy`, and `signal`, the structure follows layered packages: `api`, `application`, `domain` (where applicable), and `infrastructure`. The dependency direction remains explicit: `api` depends on `application`; `application` coordinates domain rules and infrastructure adapters; and `infrastructure` integrates persistence and external systems. The `config` package wires cross-cutting concerns such as security, exception handling, policy configuration, and signal pipeline configuration. Dashed arrows represent UML dependency relationships between packages.

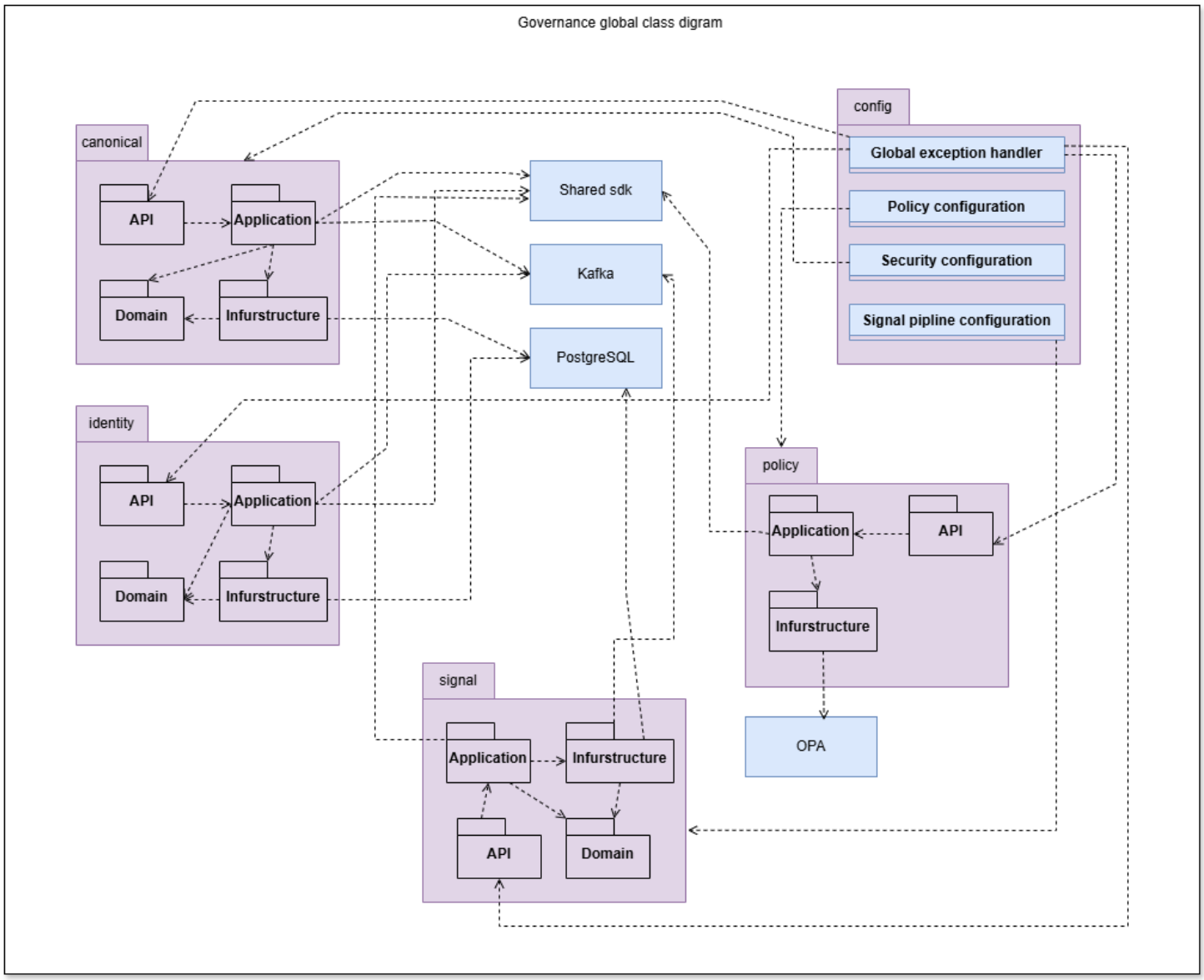

Figure 3.2: Governance service package dependency diagram

Policy Package Class Diagram

The policy package is detailed because it represents the authorization decision flow implemented in the backend. The API layer receives policy evaluation requests, the application layer builds and evaluates policy context, and the infrastructure layer communicates with OPA for decision execution. Shared SDK contracts standardize request and response models across services. This diagram emphasizes that authorization is delegated to the governance policy mechanism rather than enforced only in frontend logic.

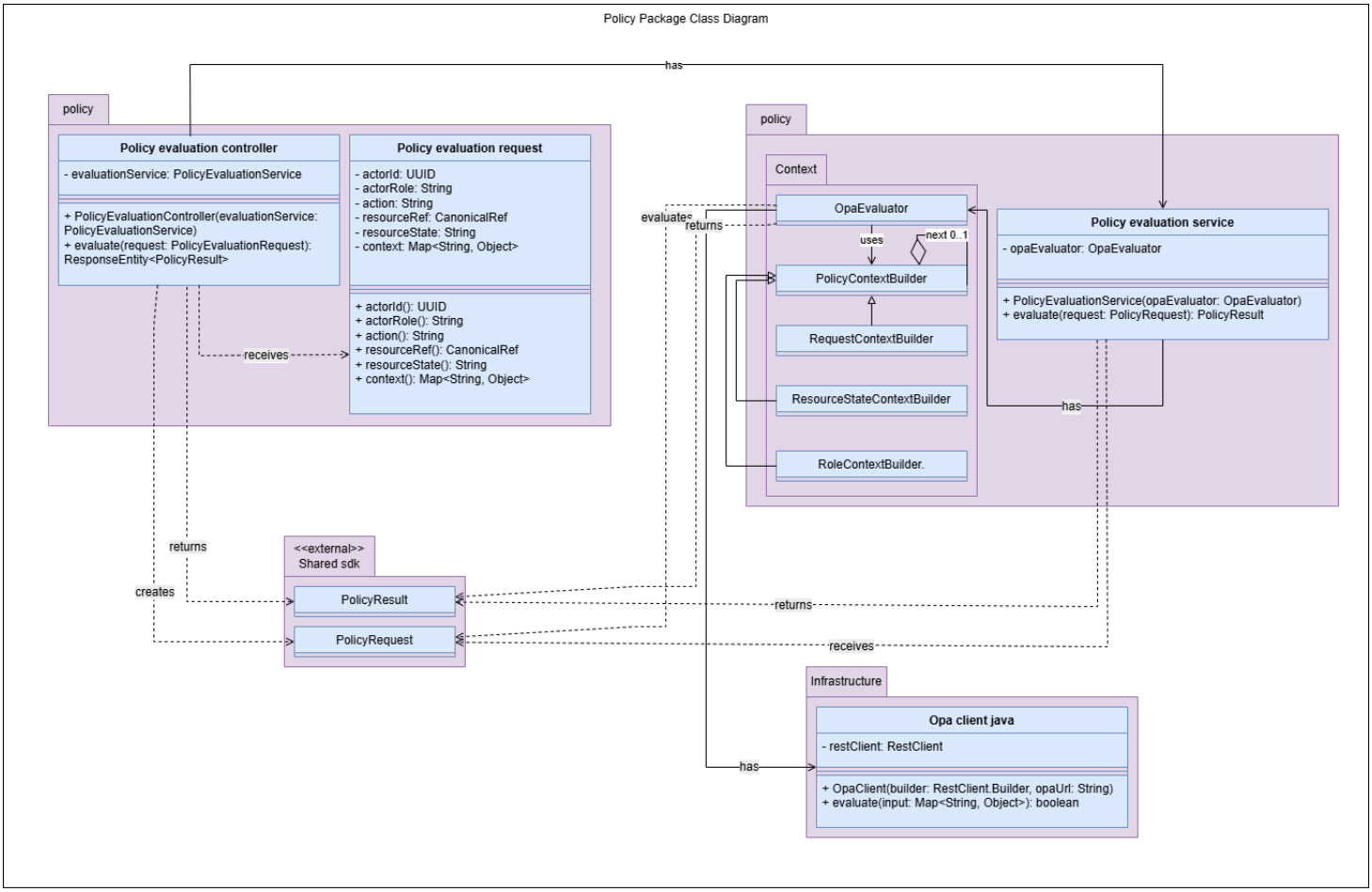

Figure 3.3: Policy package class diagram

Signal Package Class Diagram

The signal package is detailed because it captures event intake, processing, routing, notification fan-out, and audit/event propagation. Its internal organization follows a processing pipeline (chain) structure and integrates with Kafka messaging and audit persistence. This view complements the package dependency diagram by exposing the internal signal-processing collaboration.

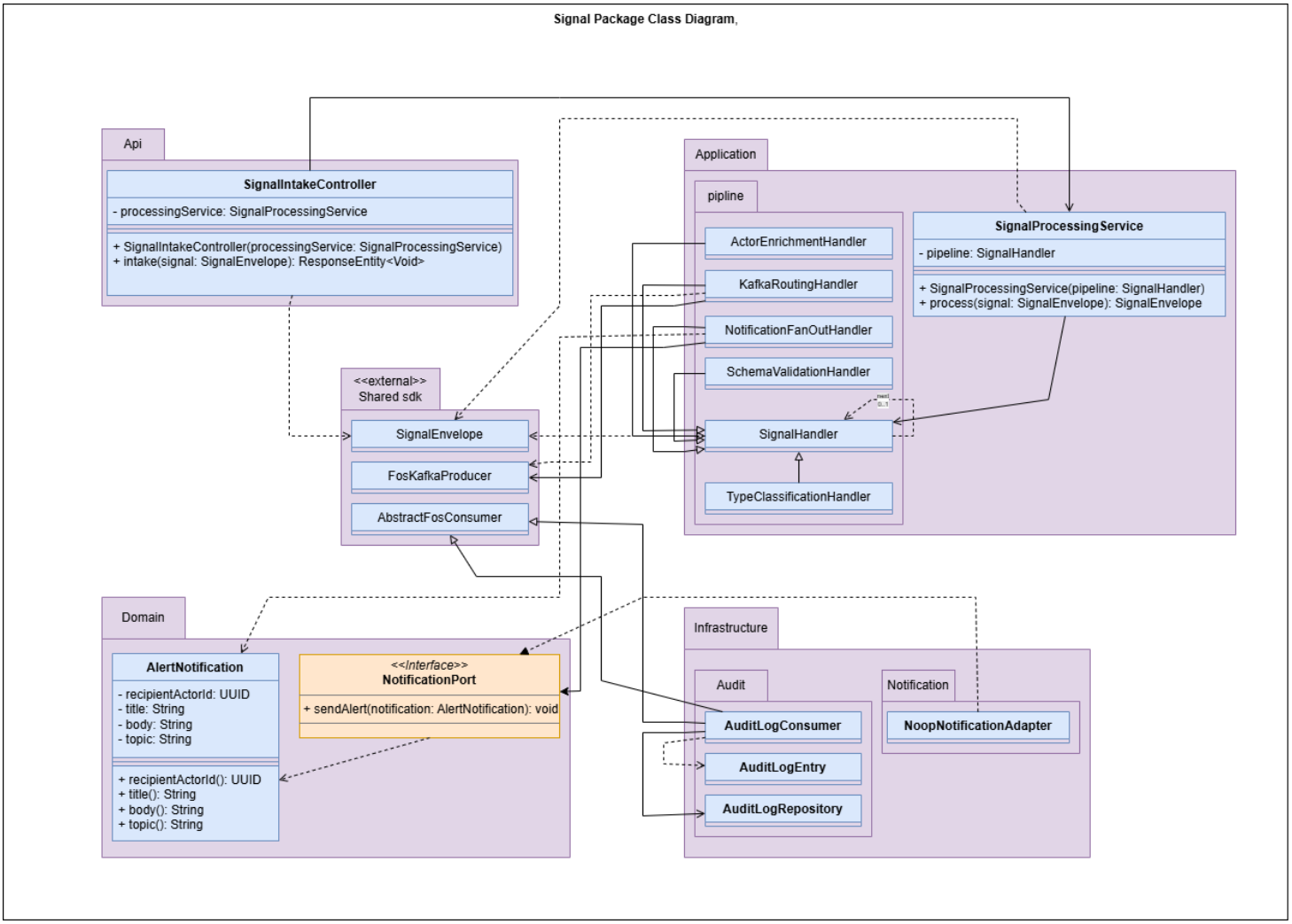

Figure 3.4: Signal package class diagram

Identity Package Class Diagram

The identity package is detailed because it manages platform actors, actor roles, lifecycle operations, and identity synchronization. It links authentication identity context with governance-level actor representation. The diagram focuses on the most relevant identity classes and relationships rather than listing every technical file.

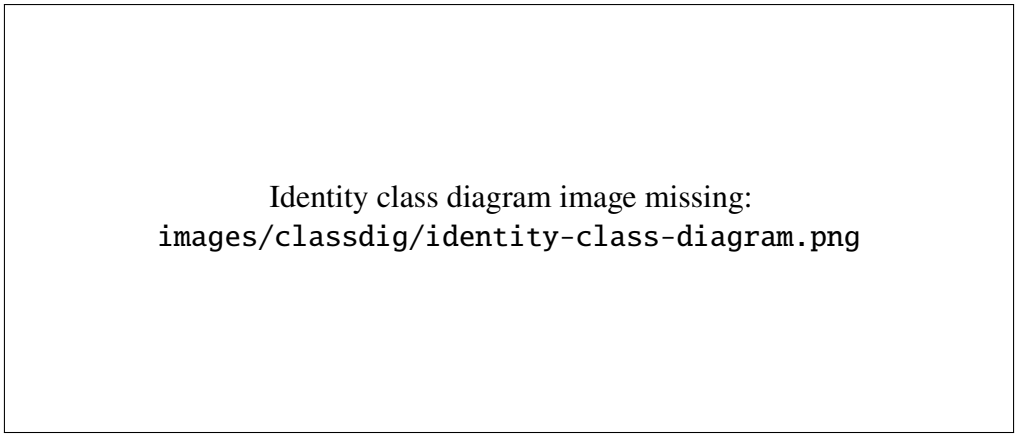

Figure 3.5: Identity package class diagram

The following diagrams shift from governance control-plane internals to workspace-side data structures, specifically document management and calendar/event management.

3.3.6 Document Management Class Diagram

This diagram presents how documents are modeled through metadata, version history, and storage links. It separates business metadata from file content kept in object storage.

Figure 3.6 highlights that WorkspaceDocument is the main record, while DocumentVersion stores each file revision with bucket and object-key references. DocumentCategory and DocumentState structure classification and lifecycle control.

Document class diagram image missing:
`images/classdig/document-class-dig.png`

Figure 3.6: Document management class diagram

3.3.7 Calendar and Event Class Diagram

This diagram explains how workspace events are modeled, including attendees, required documents, and staff tasks linked to each event.

Figure 3.7 shows that WorkspaceEvent stores core event information (title, type, schedule, location, and description), while AttendeeRef, RequiredDocument, and TaskAssignment capture participation and operational follow-up. EventType and EventState define classification and lifecycle, and delete actions are handled as archive operations.

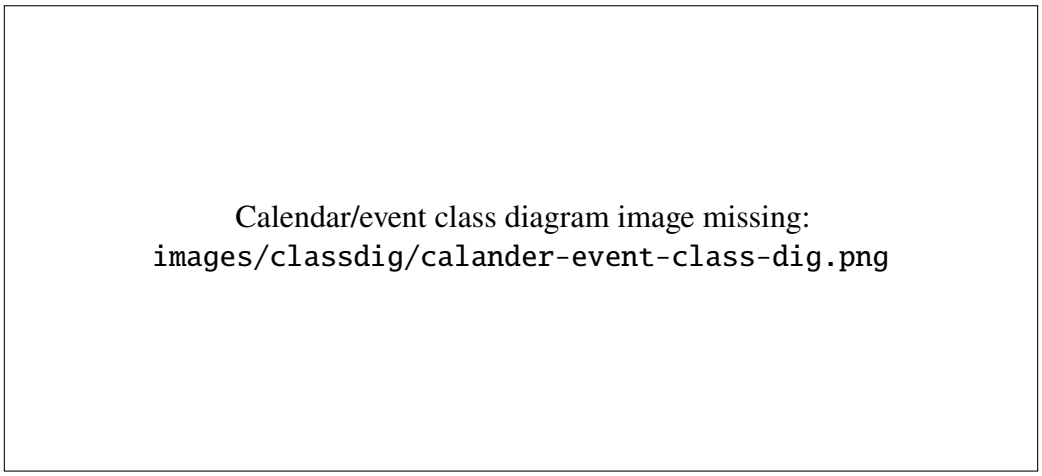

Figure 3.7: Calendar and event class diagram

3.3.8 Storage Mapping

Table 3.9: Storage mapping for Football OS

Storage Element	Used For
PostgreSQL	Governance structured data such as actors, roles, players, teams, and audit logs
MongoDB	Workspace aggregates such as documents, document versions, events, and notifications
MinIO	Actual uploaded files and edited document versions
Keycloak	Authentication and identity provider data
OPA	Authorization policy rules
Kafka	Asynchronous events and operational signals between components
Redis	Gateway rate limiting and request-throttling state

Table 3.9 summarizes storage responsibilities in Football OS, while Subsection 3.3.4 explains cross-service references and consistency handling across these stores.

The player profile is not represented as a separate class diagram in the main chapter because it is built by aggregating canonical player data, workspace documents, policy decisions, and storage download links. It is therefore explained in the processing design section.

3.4 Processing Design

This section analyzes four core processing flows used by the platform.

3.4.1 Authentication Process Flow

This flow covers authentication and token-based access across the user, frontend, Gateway, identity provider, and backend services.

Figure 3.8 highlights the security boundary created by the API Gateway. The frontend never calls internal services directly; instead, authenticated requests pass through the Gateway, where token validation and actor-context propagation are centralized. This flow reduces duplicated authentication logic in downstream services while keeping protected backend APIs behind a single controlled entry point.

The authentication flow is executed through the following steps:

1. The user requests access from the frontend.
2. The frontend redirects the user to the identity provider.
3. The user authenticates and receives a token-based session.
4. The frontend calls backend APIs through the Gateway with the token.
5. Backend services validate the authenticated context before processing.

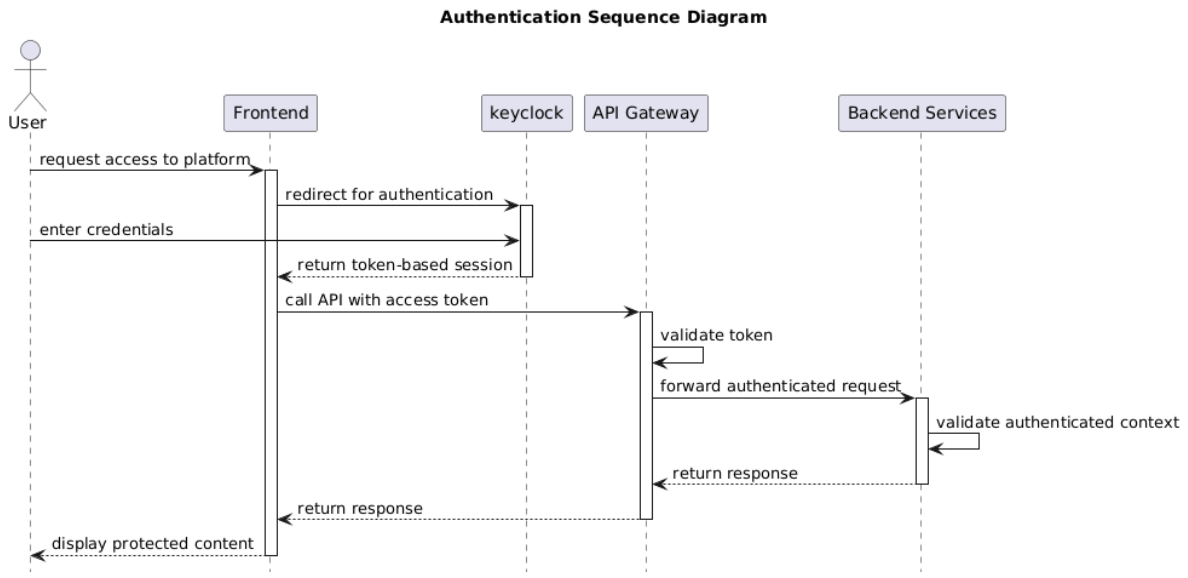

Figure 3.8: Authentication sequence diagram

Exception / alternative flow: if the token is invalid, expired, or missing, the request is rejected and access is denied.

3.4.2 Document Management Flow

This flow covers document upload, access, and online editing.

Figure 3.9 highlights the critical path of document management. Authorization is performed before storage or editor access is generated, which prevents unauthorized users from obtaining file URLs or editor configurations. The separation between MongoDB metadata and MinIO binary storage also preserves version history without storing large files directly inside the database.

The flow executes through the following steps:

1. The user uploads or opens a document through the frontend and Gateway.
2. Workspace verifies authorization through Governance/Policy.
3. For upload, Workspace generates a storage URL and the file is stored in MinIO.
4. Metadata and version information are stored in MongoDB.
5. Online viewing/editing is handled through OnlyOffice, and edited content is saved as a new version.

Exception / alternative flow: if policy denies the action, no storage URL is generated and the document operation is refused.

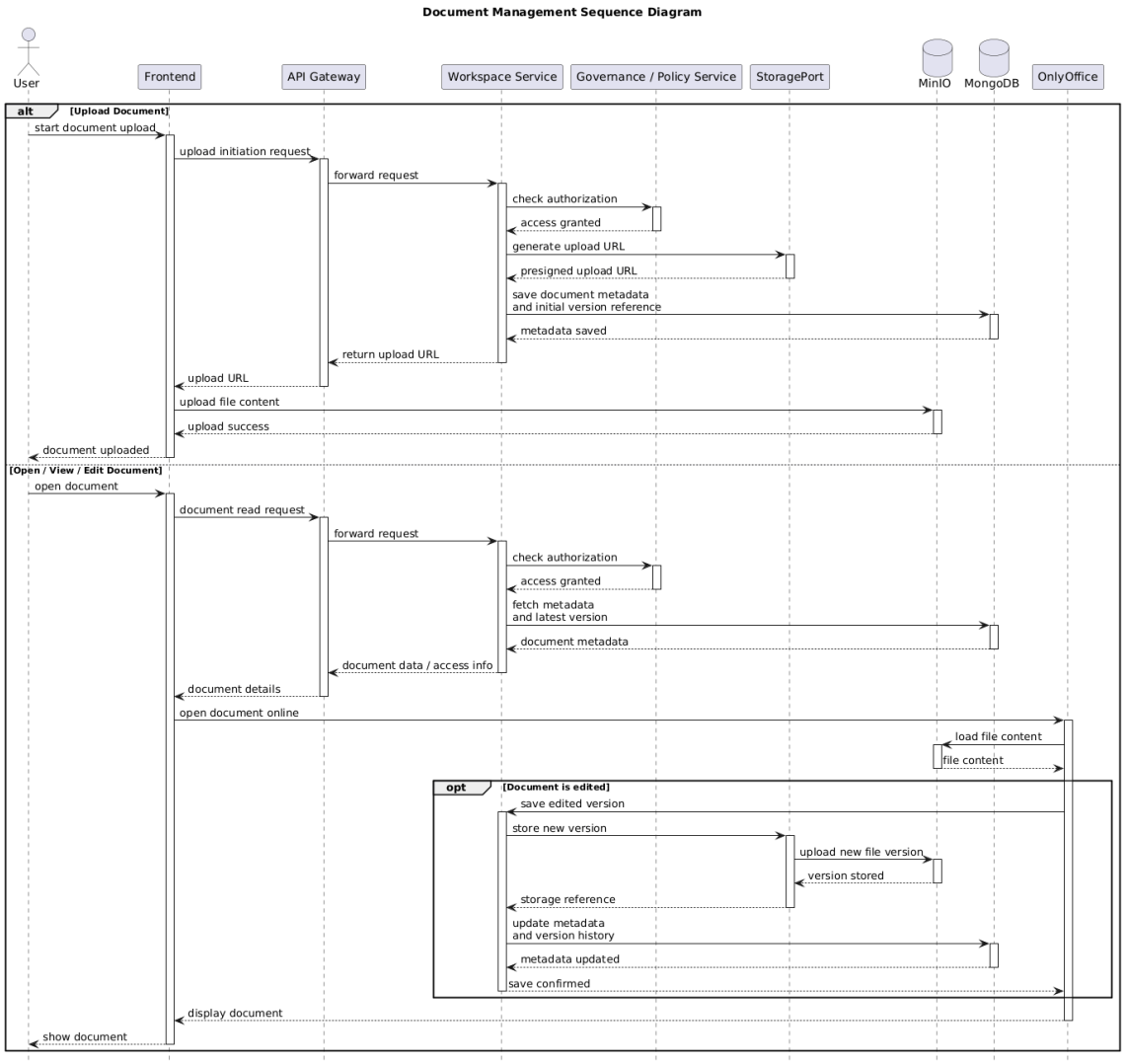

Figure 3.9: Document management sequence diagram

3.4.3 Calendar and Event Management Flow

This flow covers event creation, update, archival, and notification generation.

Figure 3.10 emphasizes that event operations are not limited to simple calendar updates. Each accepted action combines authorization, data validation, persistence, notification generation, and traceability. This flow is important because event changes may affect several staff members and must therefore remain consistent, visible to concerned users, and auditable.

The flow executes through the following steps:

1. An authorized user creates, updates, or archives an event through the Gateway.
2. Workspace validates data and verifies authorization.
3. Event aggregates (including attendees, tasks, and required documents) are stored in MongoDB.
4. Notifications are generated for concerned users.

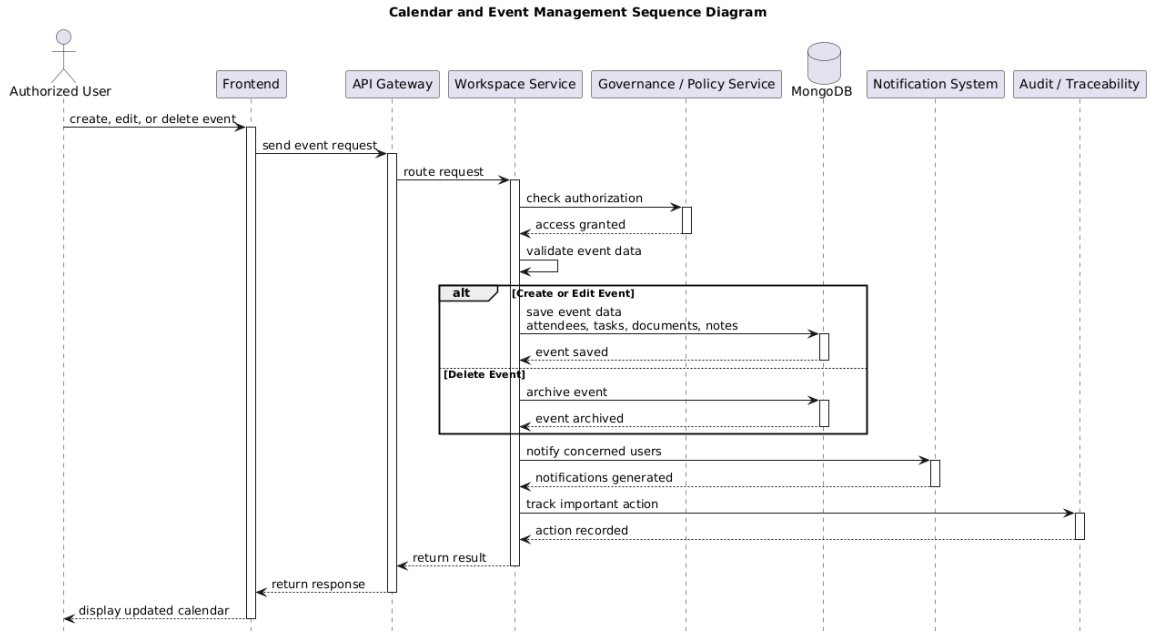

Figure 3.10: Calendar and event management sequence diagram

5. Important actions are tracked for traceability.

Exception / alternative flow: if the user is not authorized, the event is not created, modified, or archived.

In this design, delete operations are implemented as archive actions rather than hard deletion.

3.4.4 Player Profile Consultation Flow

This flow covers how the platform builds a secured player profile.

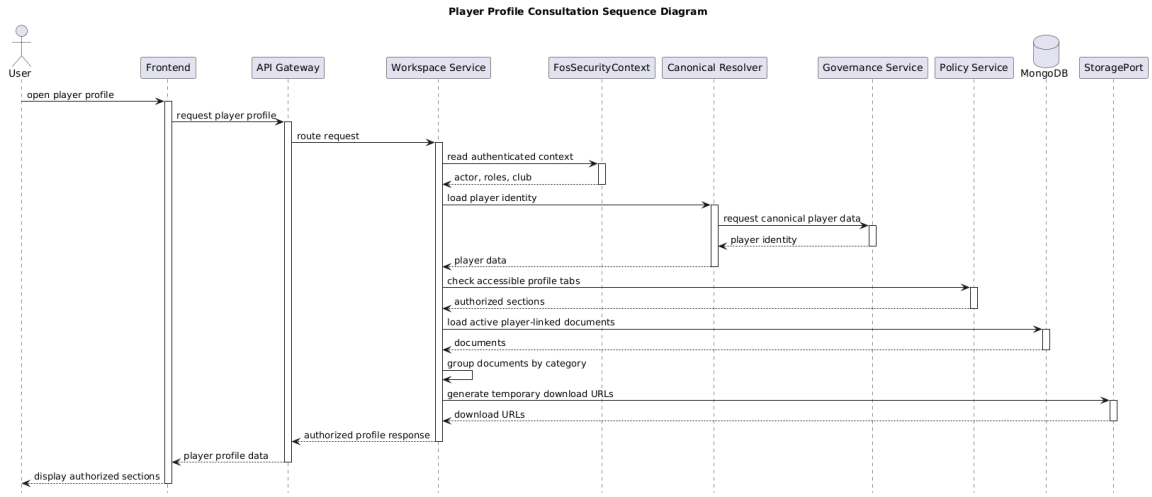

Figure 3.11: Player profile consultation sequence diagram

Figure 3.11 shows that player-profile consultation is a filtered aggregation process rather than a direct database read. The Workspace Service combines canonical player identity, workspace-linked documents, storage references, and policy decisions before returning the response. This

protects sensitive sections, especially medical information, by omitting unauthorized tabs and documents before the data reaches the frontend.

The flow executes through the following steps:

1. The user requests a player profile through the frontend and Gateway.
2. Workspace reads authenticated context and resolves canonical player identity.
3. Workspace evaluates tab and section permissions through policy checks.
4. Authorized player-linked documents are loaded and grouped by category.
5. The response returns only permitted sections with temporary download links when needed.

Exception / alternative flow: if policy denies sensitive sections, restricted tabs are hidden and related data is omitted from the response.

3.5 Security and Authorization Design

Football OS combines identity-provider authentication, gateway-level token validation, policy-based authorization, and backend-side access checks. The security model is designed so that the frontend improves usability but does not become the final enforcement point for protected resources [6, 21, 22, 20, 7].

3.5.1 Security Implementation Details

Table 3.10: Security implementation choices

Security concern	Implementation choice
Authentication	Keycloak with OIDC and JWT-based access tokens.
Token validation	API Gateway validates access tokens using Keycloak JWKS through the resource-server security configuration.
Session model	Backend services remain stateless and rely on token-based authentication context.
Token propagation	The frontend sends a bearer token to the Gateway; the Gateway enriches/forwards actor context to downstream services.
Authorization	Workspace calls the Governance policy endpoint, which relies on OPA-backed decisions.
Rate limiting	Gateway-level Redis-backed rate limiting protects backend entry points from excessive requests.
File access	Document metadata and category are checked by policy before storage or editor access is generated.
Upload protection	Upload actions are constrained by authorization rules, document category, and role-based visibility.
CSRF exposure	Exposure is reduced because API calls use bearer-token authorization rather than cookie-based backend sessions.
CORS policy	In deployment, allowed origins should be restricted to the configured frontend origin.

This design keeps security enforcement on the server side. Hidden buttons and role-based interface states improve the user experience, but protected actions are still checked by backend

services before data, storage URLs, or editor configurations are returned.

Table 3.11: Access-control matrix for core platform actions

Resource / Action	Club Administrator	Head Coach	Coaching Staff	Medical Staff
Manage users and roles	Allowed	Denied	Denied	Denied
Manage canonical players/teams	Allowed	Policy	Denied	Denied
Create/update/archive events	Denied	Allowed	Policy	Denied
Consult events	Allowed	Allowed	Allowed	Policy
Upload normal documents	Allowed	Allowed	Policy	Policy
Consult normal documents	Allowed	Allowed	Policy	Policy
Access medical documents	Restricted	Denied	Denied	Allowed
Access admin/contract documents	Allowed	Denied	Denied	Denied
Consult player profiles	Allowed	Allowed	Policy	Restricted
Receive notifications	Allowed	Allowed	Allowed	Allowed

Table 3.11 summarizes the nominal access posture by role. In operational execution, final authorization remains policy-based on the backend.

Conclusion

This chapter examined the design of Football OS through global architecture, backend modules, data models, and core processing flows. The selected diagrams show how Gateway, Governance, Workspace, and SDK work together to support document management, calendar events, and player-profile consultation.

Chapter 4: Implementation and Realization

Introduction

This chapter presents the implementation state of Football OS at the end of the internship period. It covers the software environment, the implemented solution, the main graphical interfaces, and the deployment and integration setup.

4.1 Development Environment

4.1.1 Software Environment

This subsection summarizes the software stack used to implement and run Football OS during the internship period.

4.2 Implementation Overview

Football OS was implemented as a distributed backend platform connected to an Angular workspace frontend through the API Gateway. The implementation provides an operational baseline for protected access, document workflows, calendar/events, notifications, search, player profiles, and OnlyOffice-based document editing. Tables 4.1 and 4.2 summarize the technical stack and delivered scope.

Implementation details and technology usage remain aligned with the official documentation references cited in this report.

4.2.1 Gateway and Secured Routing

The frontend sends API requests to port 8080 (Gateway) instead of directly calling internal services. The Gateway centralizes entry control, propagates authenticated context, and routes workspace endpoints to Workspace Service and actor/canonical/policy endpoints to Governance Service.

Table 4.1: Software environment used for implementation

Technology	Version	Use in project
Java	21	Backend language for Gateway, Governance, Workspace, and SDK modules.
Node.js	20+	Frontend runtime and build environment for Angular development.
Angular	Project	Frontend framework for the workspace application.
TypeScript	Project	Typed language for frontend development.
Tailwind CSS	Project	Utility-first CSS framework for interface styling.
Spring Boot	Project	Backend framework for the API Gateway and domain services.
PostgreSQL	Project	Relational database for governance data (actors, roles, players, teams, audit).
MongoDB	Project	Document database for workspace aggregates (documents, events, notifications, metadata).
Keycloak	Project	Identity and access management for authentication and user sessions.
Open Policy Agent	Project	Policy engine used to evaluate authorization rules.
Apache Kafka	Project	Messaging platform for asynchronous events and operational signals.
MinIO	Project	Object storage for uploaded files and edited document versions.
OnlyOffice	Project	Online document editor for viewing and editing workspace documents.
Docker / Docker Compose	Project	Local multi-service deployment and infrastructure orchestration.
Git / GitHub	Project	Version control and source-code collaboration.
Visual Studio Code / IntelliJ IDEA	Project	Development environments for frontend and backend.
PlantUML / Draw.io	Project	Tools used to create UML and architecture diagrams.

Table 4.2: Implementation overview by feature

Feature	Backend support	Frontend support	Status
Authentication and secured access	Token validation and secured routing with identity propagation.	Login flow and protected routes.	Implemented
Document management	Document APIs, policy checks, and metadata lifecycle.	Authorized document list/detail workflows.	Implemented
Document versioning	Version entities and storage references.	Version access in document workflows.	Implemented
OnlyOffice integration	Editor configuration and save callbacks.	Embedded editor launch from document actions.	Implemented
Calendar/event management	Event APIs with validation and authorization.	Calendar and event-detail interfaces.	Implemented
Player profile consultation	Canonical/workspace aggregation with policy filtering.	Profile consultation and tab navigation.	Partial
Notifications	Notification generation and retrieval services.	Notification display and state updates.	Implemented
Search	Search endpoints with access-aware filtering.	Search entry points in workspace modules.	Implemented
Authorization integration	Governance policy decisions and backend checks.	UI gating by role and permission context.	Implemented
Storage integration	MongoDB metadata and MinIO object storage separation.	Upload/open workflows via backend-issued links.	Implemented

4.2.2 Document Storage and Versioning

Document metadata (category, state, ownership, version references) is stored in MongoDB, while binary file content is stored in MinIO. Versions are represented as separate records linked to a logical document. Online edits can create a new version and update related metadata, preserving document history.

In a typical upload flow, Workspace verifies permissions through Gateway, stores binary content in MinIO, persists metadata and version references in MongoDB, and archives by state transition rather than hard deletion.

4.2.3 OnlyOffice Integration

Workspace generates the OnlyOffice editor configuration in a controlled context. The document is opened through secure backend-managed access parameters. Save callbacks update version metadata and content references. Access remains enforced by backend policy decisions [12].

When a user opens or saves a document, Workspace verifies authorization before issuing editor configuration and registers callback results as a new version.

4.2.4 Event Workflow Example

A typical event workflow starts when the Head Coach creates an event from the calendar interface. The frontend sends the request through the Gateway to the Workspace Service. Workspace validates the event data, checks authorization, then stores the event aggregate in MongoDB with its schedule, attendees, tasks, and required documents. If the event affects other users, notification entries are generated so that concerned staff members can consult the update from the workspace.

4.2.5 Technical Challenges Encountered

The implementation phase also involved several integration challenges caused by the distributed nature of the platform. These issues were useful for validating the architectural choices because they required stable service communication, consistent identity propagation, policy debugging, and coordination between metadata and binary storage.

Table 4.3: Technical challenges encountered during implementation

Challenge	Impact	Resolution
Docker networking	Services needed stable local communication across containers and local processes.	Standardized local ports and routed frontend traffic through the API Gateway.
Keycloak role mapping	Multiple emitted roles could affect actor-context extraction and authorization behavior.	Role extraction was normalized in the backend security context.
OnlyOffice callback handling	Editor save operations required backend-managed versioning instead of direct file replacement.	Save callbacks were linked to document version creation and metadata updates.
Policy debugging	Access rules had to match role, action, resource type, and document category combinations.	Policy checks were isolated through Governance and OPA-backed decisions.
Multi-storage design	Metadata and binary files had to remain separated while preserving traceability.	MongoDB stores document metadata and version references, while MinIO stores binary file versions.

These challenges confirmed the need for clear service boundaries and reusable SDK abstractions, especially for security context, policy evaluation, storage access, and event propagation.

4.3 Main Graphical Interfaces

4.3.1 Authentication Interface

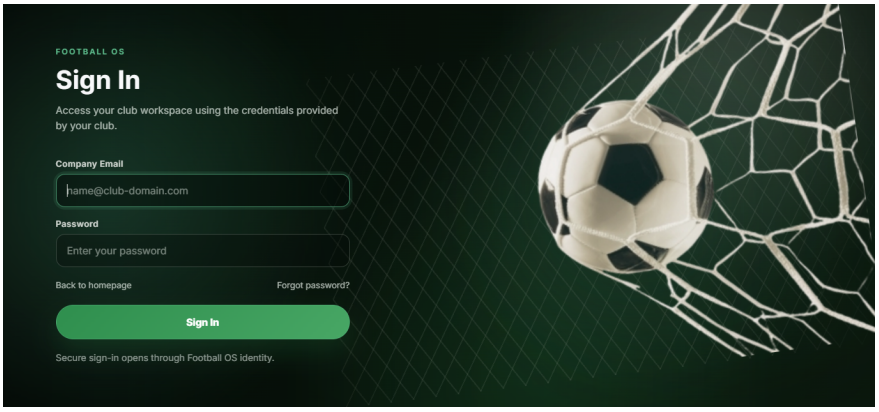

Figure 4.1: Authentication interface

Figure 4.1 represents the secure entry point of Football OS. Users must authenticate before accessing protected workspace features. The authenticated session is then used for backend request authorization. After successful authentication, users are redirected to the calendar interface, which acts as the workspace landing page.

4.3.2 Document Management Interface

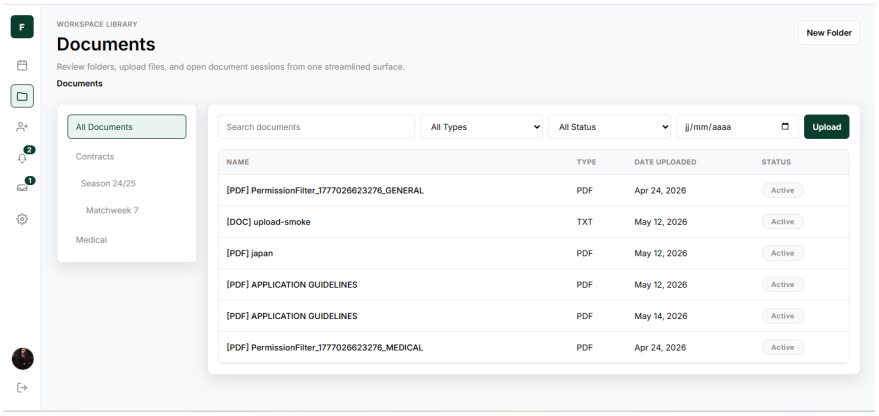

Figure 4.2: Document management interface

Figure 4.2 shows document consultation and upload workflows. This interface allows authorized users to consult, upload, organize, and archive documents. Access to documents depends on the user role and document category.

4.3.3 Online Document Viewer and Editor

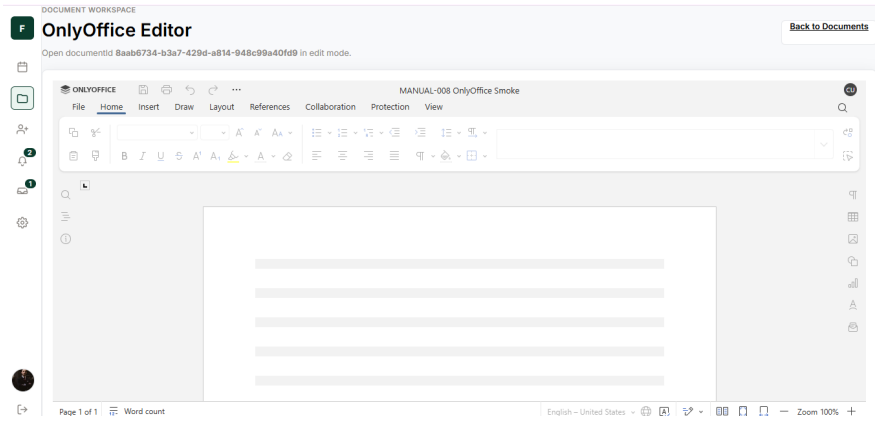

Figure 4.3: OnlyOffice editor interface

Figure 4.3 shows the OnlyOffice integration used to view and edit documents online. The platform controls access, while document content is loaded through temporary storage links.

4.3.4 Calendar Interface

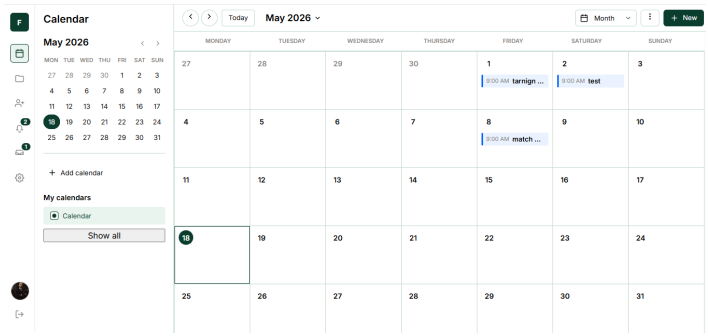

Figure 4.4: Calendar interface

Figure 4.4 is the default landing page after authentication. It allows users to view club events and activities. Authorized users can create, edit, or archive events according to their role and policy rules. Event details include schedule, location, attendees, tasks, notes, and required documents.

4.3.5 Player Profile and Notifications

Player-profile consultation remains available in the workspace and displays role-filtered operational and document-related information, especially for sensitive medical or administrative categories. Operational notifications are also accessible from the workspace interface for updates such as event changes and task assignments.

4.4 Deployment and Integration

Football OS was tested in a local Docker-based environment. The deployment includes backend services and infrastructure containers. This setup allows integration testing of routing, authorization, storage, messaging, and document editing workflows.

Table 4.4: Deployment and integration components

Service	Port	Role
API Gateway	8080	Routes frontend requests to backend services.
Governance Service	8081	Manages users, roles, permissions, canonical data, and audit.
Workspace Service	8082	Manages documents, events, notifications, search, and player profiles.
Workspace Frontend	4200	Angular user interface; calls backend APIs through the Gateway.
PostgreSQL	5432	Stores governance structured data.
MongoDB	27017	Stores workspace data.
MinIO	9000/9001	Stores document files and object administration data.
Keycloak	Docker Compose	Provides authentication.
OPA	8181	Evaluates authorization policies.
Kafka	9092	Supports asynchronous event communication.
OnlyOffice	Docker Compose	Provides document viewing and editing.

As summarized in Table 4.4, frontend API calls are sent to port 8080 and are not sent directly from the frontend to the Workspace Service.

Conclusion

This chapter detailed the software environment, implementation overview, main graphical interfaces, and deployment/integration of Football OS. The implemented product provides secured access to workspace features such as document management, calendar events, player profile consultation, notifications, search, and online document editing.

Chapter 5: Testing and Validation

Introduction

This chapter summarizes the testing and validation activities performed for Football OS.

5.1 Testing Strategy

The testing approach combined complementary levels: smoke checks for service health, functional and integration tests for core workflows across Gateway, Governance, Workspace, and infrastructure dependencies, and security checks for protected routes and role restrictions, completed by focused manual UI validation of principal frontend flows.

5.2 Test Environment

Table 5.1 presents the integrated validation environment used during testing.

Table 5.1: Components involved in the test environment

Component	Role in testing
Docker Compose	Starts local services and dependencies.
API Gateway	Routing and protected entry-point checks.
Governance Service	Roles, permissions, canonical data, and policy-call checks.
Workspace Service	Documents, events, notifications, search, and profile checks.
PostgreSQL	Persistence checks for governance and audit data.
MongoDB	Persistence checks for workspace aggregates and metadata.
MinIO	Binary document storage and retrieval checks.
Keycloak	Authentication and identity-context checks.
OPA	Policy decision evaluation for authorization cases.
Kafka	Validation of asynchronous event/signal behavior.
OnlyOffice	Online document opening/editing callback checks.
Angular frontend/browser	Manual user workflow and interface checks.

5.3 Test Cases

Table 5.2 summarizes representative scenarios and their current validation status.

These tests were executed in a local Docker-based environment using health checks, authenticated frontend workflows, and backend responses through the API Gateway. The objective

Table 5.2: Representative test cases and current validation status

ID	Test Scenario	Type	Expected Result	Status
T01	Gateway health check	Smoke	Gateway is healthy.	Passed
T02	Governance health check	Smoke	Governance is healthy.	Passed
T03	Workspace health check	Smoke	Workspace is healthy.	Passed
T04	User login with valid account	Functional	Session is created.	Passed
T05	Unauthenticated protected access	Security	Access is denied.	Passed
T06	Upload document	Integration	File and metadata stored.	Passed
T07	Open document in OnlyOffice	Integration	Editor opens securely.	Partial
T08	Archive document	Functional	Document is archived.	Passed
T09	Create event as Head Coach	Functional	Event is persisted.	Passed
T10	Event creation by unauthorized staff	Security	Policy denies creation.	Passed
T11	Role-filtered player profile consultation	Integration	Permitted sections only.	Manual
T12	Notification consultation	Functional	Notifications accessible.	Manual
T13	Workspace search	Functional	Accessible results returned.	Partial
T14	Medical document access control	Security	Medical access restricted.	Partial

was integration and role-based-access validation in development conditions, not production benchmarking.

5.4 Performance Evaluation

In addition to functional validation, a small set of latency observations was collected in the local Docker-based environment. These measurements provide an indicative view of integration behavior and relative operation cost. They are not production benchmarks because they were executed on a local machine with development configuration and a limited dataset.

Table 5.3: Local performance observations

Operation	Average Time	Observation
Gateway health check	35 ms	Confirms service availability.
Governance health check	42 ms	Confirms service reachability.
Workspace health check	48 ms	Confirms service reachability.
Event creation	210 ms	Includes validation and MongoDB persistence.
Document metadata creation	260 ms	Excludes large binary file transfer.
Document upload	1.8 s	Depends on file size and MinIO transfer.
OnlyOffice config generation	320 ms	Includes authorization and config generation.
Workspace search	180 ms	Measured on a local sample dataset.

Gateway and service-health operations remain below 50 ms in the local environment, while persistence, authorization, storage, and editor operations require more time. Document upload is the most expensive operation due to file transfer and object-storage interaction; broader load testing remains necessary before production deployment.

5.5 Validation Summary

Validation confirmed coherent integrated behavior across platform services, with core workflows checked for authentication, document lifecycle, events, notifications, profile consultation, and policy-based protection. Priority improvements remain broader automated coverage, production monitoring readiness, and deeper end-to-end validation.

General Conclusion

This internship project resulted in the design and implementation of Football OS, a unified digital platform for football club operations. The objective was to reduce fragmentation caused by separate tools and to provide a secured workspace for document management, event planning, player profile consultation, notifications, and search. The final solution was built as a microservices-oriented backend organized in a monorepo, structured around the API Gateway, the Governance Service, the Workspace Service, and the Shared SDK.

The implemented platform covers the main functional scope defined in this report. Authentication and protected routing are integrated end to end, governance roles and policy decisions control sensitive actions, and document workflows are handled through version-aware metadata separated from object storage. Calendar and event operations support planning and coordinated follow-up, while player profile access is filtered by permissions. In addition, notification mechanisms improve operational visibility, and OnlyOffice integration enables authorized users to consult and edit documents online in a controlled workflow.

Beyond feature delivery, the project provided important engineering lessons. The distributed nature of the solution required careful handling of service communication, identity propagation, policy debugging, document versioning, and coordination between MongoDB, PostgreSQL, MinIO, Keycloak, OPA, and the Gateway. The architectural choice of separating edge access, governance responsibilities, workspace operations, and shared SDK abstractions improved modularity, but it also introduced integration complexity that had to be managed during implementation and validation.

The validation phase confirmed that the prototype behaves coherently in the local Docker-based environment. Functional tests, security checks, and local latency observations showed that the main workflows operate as expected for development use. However, the platform still requires broader automated testing, stronger end-to-end validation, production monitoring, load evaluation, and further UI/UX refinement before production-scale adoption. Overall, Football OS provides a solid and extensible foundation for future football-specific workflows and demonstrates how a governed digital workspace can improve coordination, traceability, and controlled collaboration inside football organizations.

Bibliography

[1] Angular Team, "Angular Documentation," Angular, 2026. [Online]. Available: https://angular.dev/. [Accessed: May 21, 2026].

[2] VMware, "Spring Boot Reference Documentation," Spring, 2026. [Online]. Available: https://docs.spring.io/spring-boot/index.html. [Accessed: May 21, 2026].

[3] Tailwind Labs, "Tailwind CSS Documentation," Tailwind CSS, 2026. [Online]. Available: https://tailwindcss.com/docs. [Accessed: May 21, 2026].

[4] Microsoft, "TypeScript Documentation," TypeScript, 2026. [Online]. Available: https://www.typescriptlang.org/docs/. [Accessed: May 21, 2026].

[5] Docker Inc., "Docker Documentation," Docker Docs, 2026. [Online]. Available: https://docs.docker.com/. [Accessed: May 21, 2026].

[6] Keycloak Contributors, "Keycloak Documentation," Keycloak, 2026. [Online]. Available: https://www.keycloak.org/documentation. [Accessed: May 21, 2026].

[7] Open Policy Agent Contributors, "Open Policy Agent Documentation," OPA, 2026. [Online]. Available: https://www.openpolicyagent.org/docs/latest/. [Accessed: May 21, 2026].

[8] MongoDB Inc., "MongoDB Documentation," MongoDB, 2026. [Online]. Available: https://www.mongodb.com/docs/. [Accessed: May 21, 2026].

[9] The PostgreSQL Global Development Group, "PostgreSQL Documentation," PostgreSQL, 2026. [Online]. Available: https://www.postgresql.org/docs/. [Accessed: May 21, 2026].

[10] MinIO, Inc., "MinIO Documentation," MinIO, 2026. [Online]. Available: https://min.io/docs/minio/container/index.html. [Accessed: May 21, 2026].

[11] The Apache Software Foundation, "Apache Kafka Documentation," Apache Kafka, 2026. [Online]. Available: https://kafka.apache.org/documentation/. [Accessed: May 21, 2026].

[12] Ascensio System SIA, "ONLYOFFICE Docs API," ONLYOFFICE, 2026. [Online]. Available: https://api.onlyoffice.com/docs/docs-api/. [Accessed: May 21, 2026].

[13] PlantUML, "PlantUML Documentation," PlantUML, 2026. [Online]. Available: https://plantuml.com/. [Accessed: May 21, 2026].

[14] JGraph Ltd., "diagrams.net," diagrams.net, 2026. [Online]. Available: https://www.diagrams.net/. [Accessed: May 21, 2026].

[15] Git Project, "Git Documentation," Git SCM, 2026. [Online]. Available: https://git-scm.com/docs. [Accessed: May 21, 2026].

[16] Object Management Group, "OMG Unified Modeling Language (OMG UML), Version 2.5.1," Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1. [Accessed: May 21, 2026].

[17] T. Erl, *Service-Oriented Architecture: Concepts, Technology, and Design*. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2005.

[18] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O'Reilly Media, 2015.

[19] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Pearson, 2017.

[20] R. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[21] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," RFC 6749, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749. [Accessed: May 21, 2026].

[22] N. Sakimura *et al.*, "OpenID Connect Core 1.0 incorporating errata set 1," OpenID Foundation, Nov. 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html. [Accessed: May 21, 2026].

[23] M. Fowler, "What do you mean by Event-Driven?," martinfowler.com, Feb. 2017. [Online]. Available: https://martinfowler.com/articles/201701-event-driven.html. [Accessed: May 21, 2026].